
Finstein

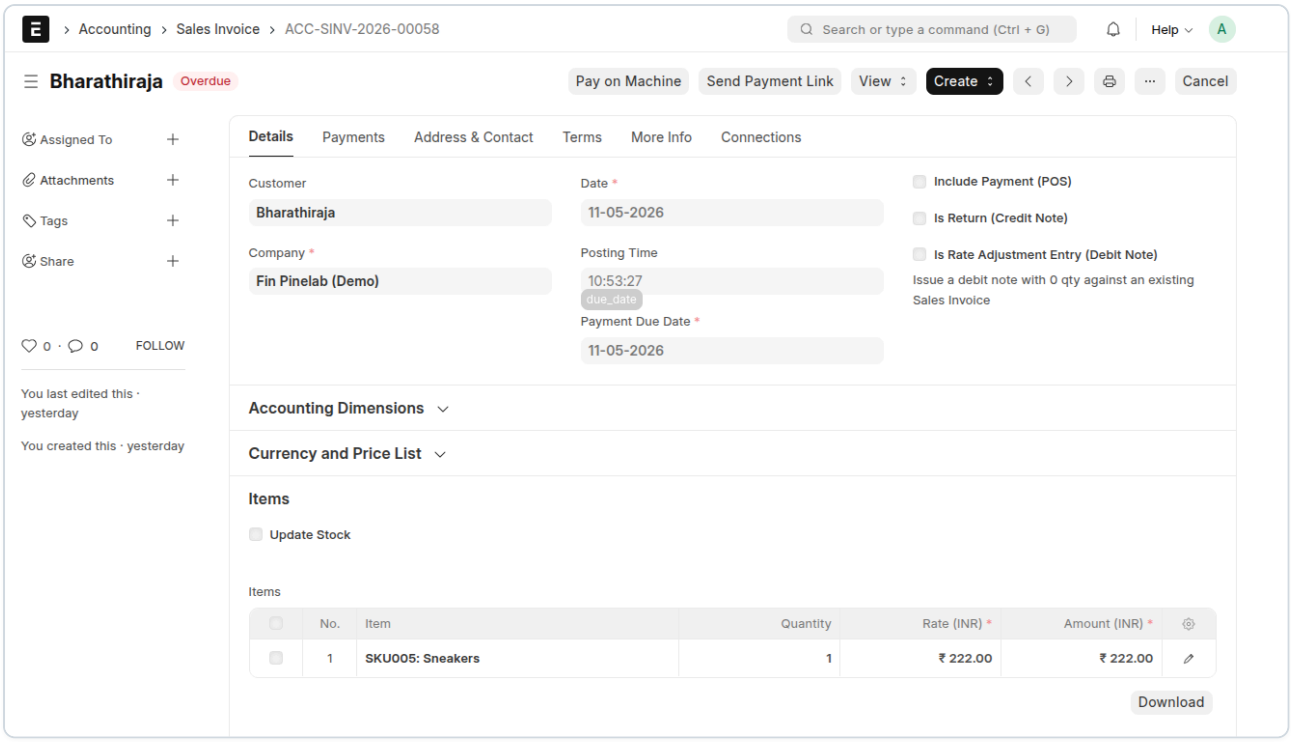
×



pinelabs_fin **Pine Labs payments inside ERPNext.** Tap-on-terminal *or* send-a-link, with the Payment Entry created automatically when the customer pays. ![ERPNext 15](https://img.shields.io/badge/ERPNext-15-blue) ![Frappe 15](https://img.shields.io/badge/Frappe-15-orange) ![License MIT](https://img.shields.io/badge/license-MIT-lightgrey)

What you get

One ERPNext app, two payment flows, zero manual reconciliation. Open any submitted invoice — you see two new buttons:



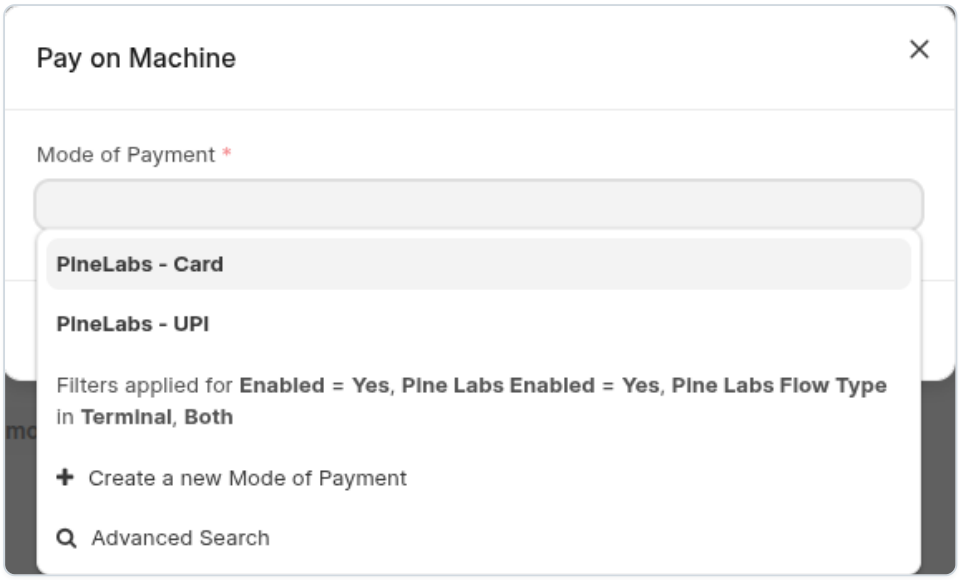
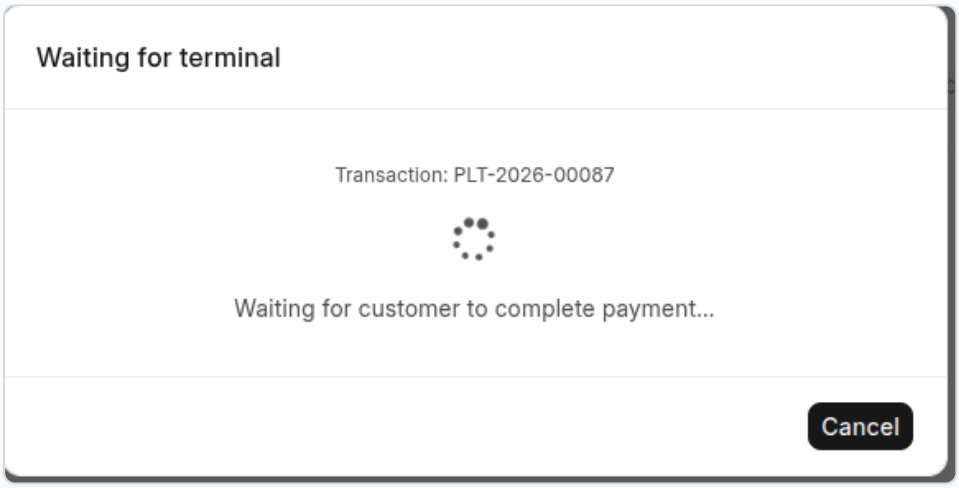
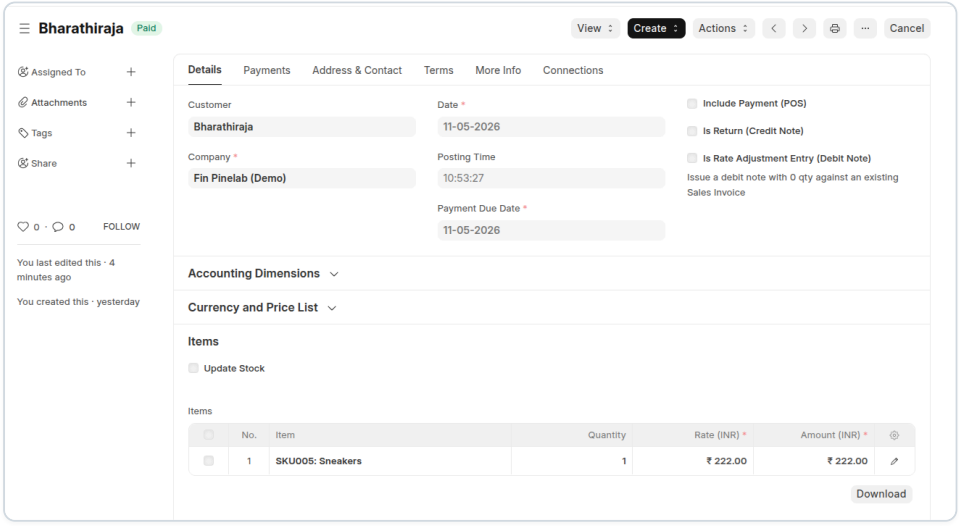
Click one. The customer pays. The invoice flips to **Paid** automatically.

Button	Use when…	Customer pays via…
Pay on Machine	The customer is in front of you	A physical Pine Labs Cloud terminal (card / UPI)
Send Payment Link	The customer is elsewhere	A Plural hosted checkout link delivered by SMS + email

> **Zero ERPNext core changes.** No custom fields are added to Sales Invoice, POS Invoice, Payment Entry, Customer, or Contact. All payment data lives in a new `Pinelabs Transaction` doctype, linked via a Dynamic Link.

Flow 1 — Pay on Machine

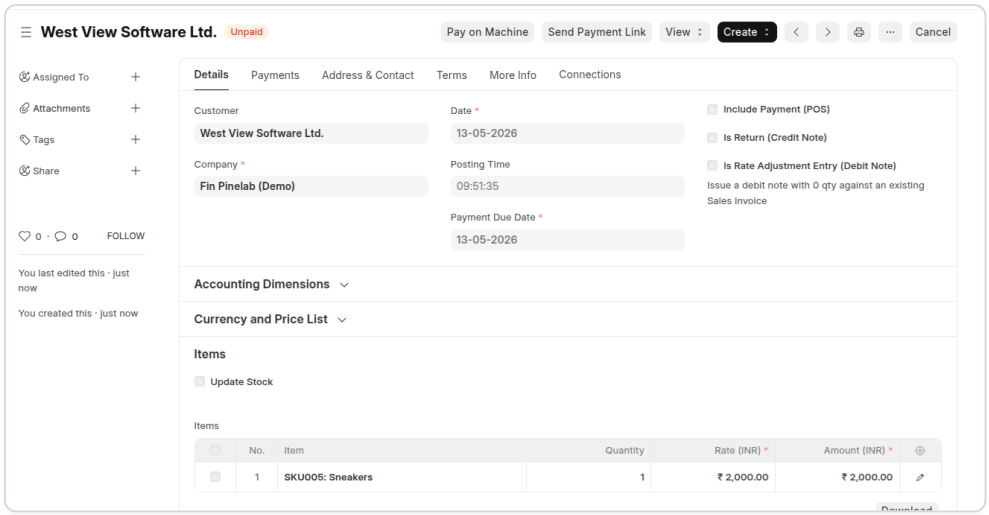
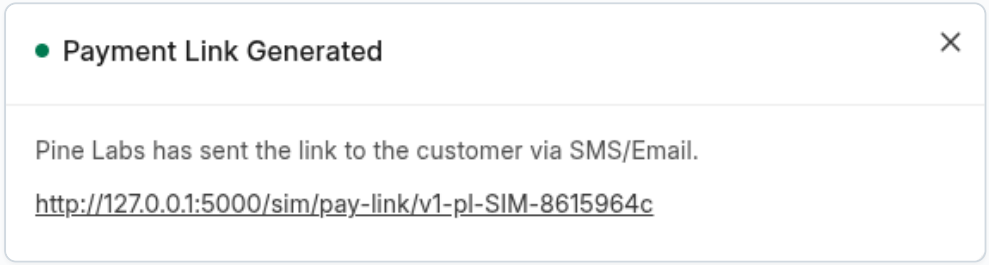
Customer at the counter. Tap card or scan UPI.

Step	What happens
1. Pick Card or UPI	
2. Wait for the customer	
3. Customer taps → invoice flips to Paid	

Total time: ~5–10 seconds. The Payment Entry is created and submitted automatically.

Flow 2 — Send Payment Link

Customer not present. Pine Labs delivers a link by SMS and email.

Step	What happens
1. Click the button	
2. Link generated and delivered	

When the customer pays — minutes or hours later — Pine Labs sends a webhook to the app. The invoice flips to **Paid** within a minute.

Quick install

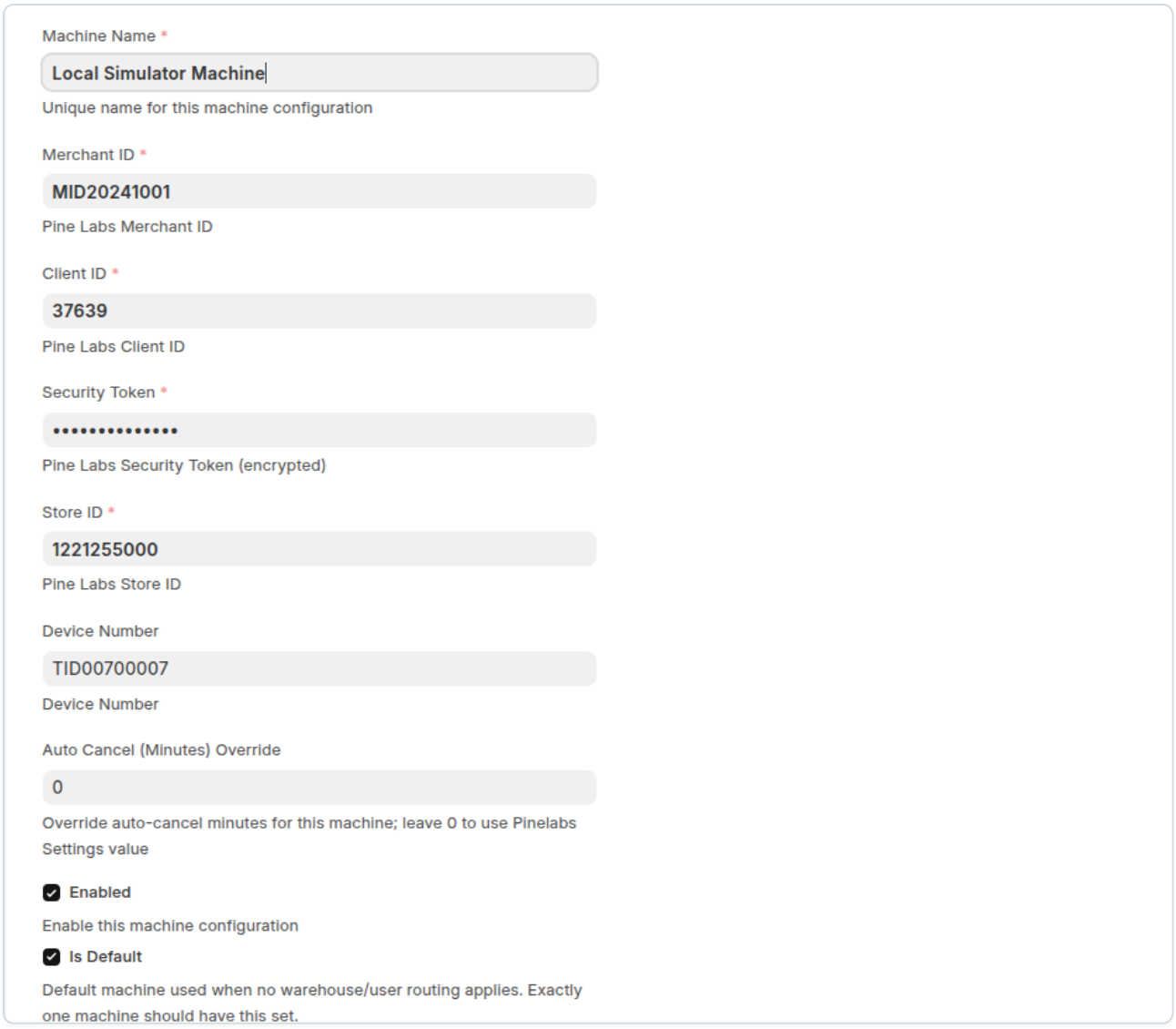
```
cd ~/frappe-bench
bench get-app https://github.com/<owner>/pinelabs_fin
bench --site <your-site> install-app pinelabs_fin
bench --site <your-site> migrate
```

Need Frappe v15, ERPNext v15, Python 3.10+, MariaDB 10.6+ with InnoDB.

Configure once

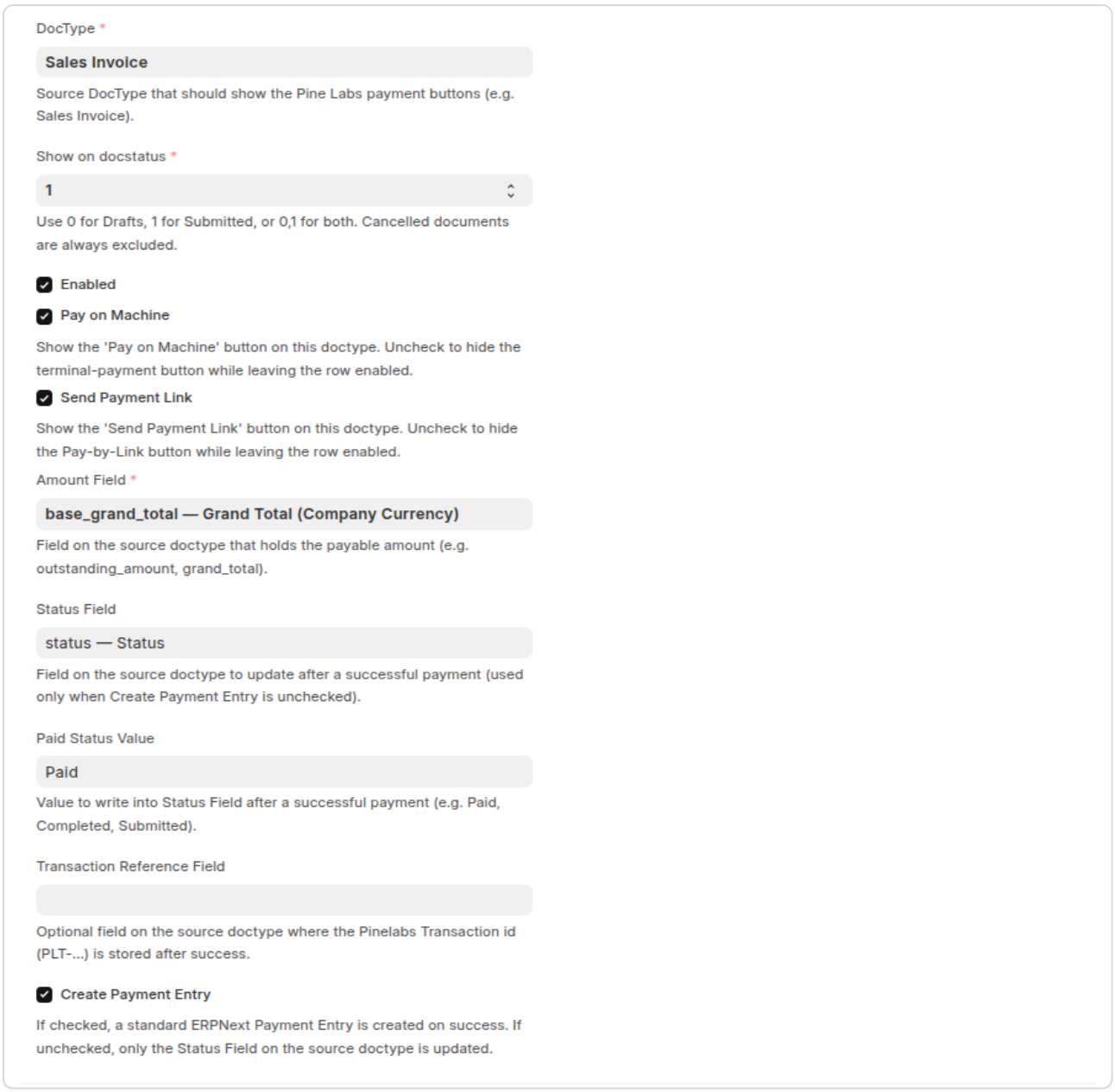
All configuration lives in a single doctype: **Pinelabs Settings**. Two things to fill in.

Your Pine Labs machine



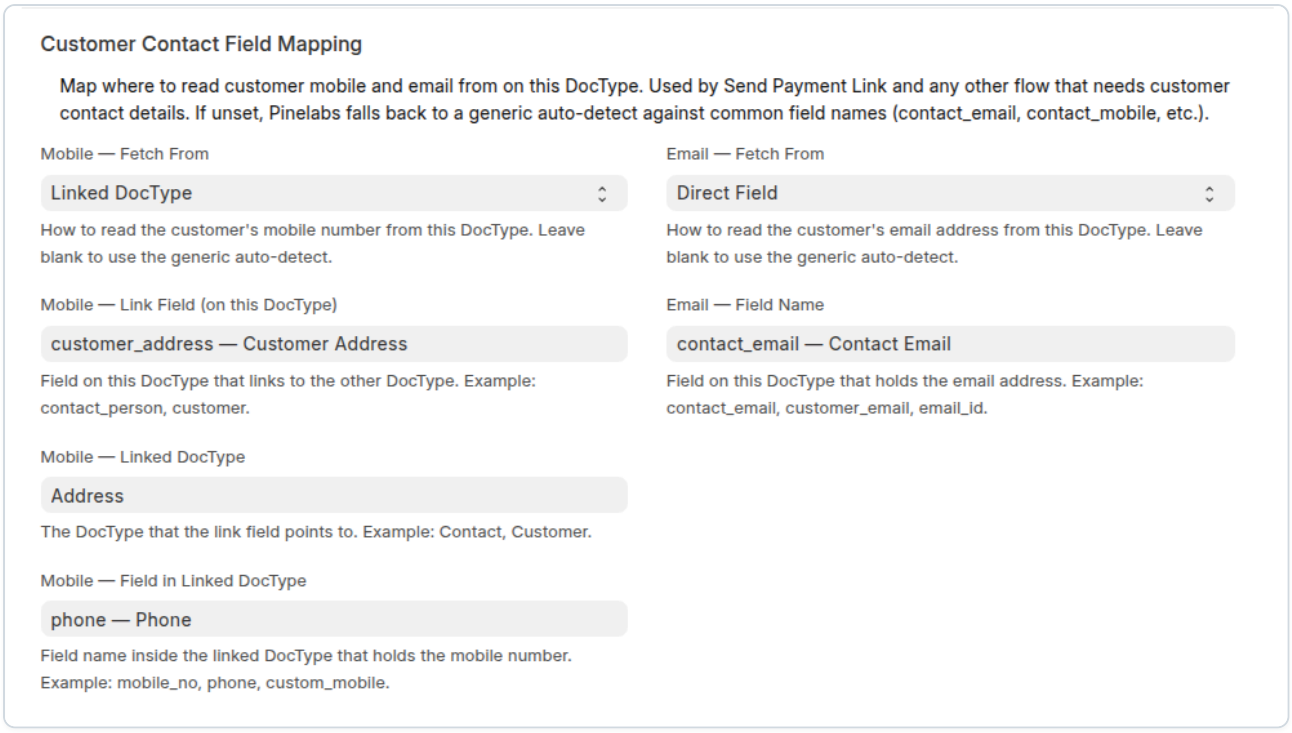
Drop in the credentials from your Pine Labs onboarding email. Mark one machine as **Is Default** and you are done.

Which doctypes can be paid



Add a row per doctype you want the buttons on (Sales Invoice, POS Invoice, Payment Request, or your own custom doctype). The two toggles **Pay on Machine** / **Send Payment Link** control which buttons render on that doctype.

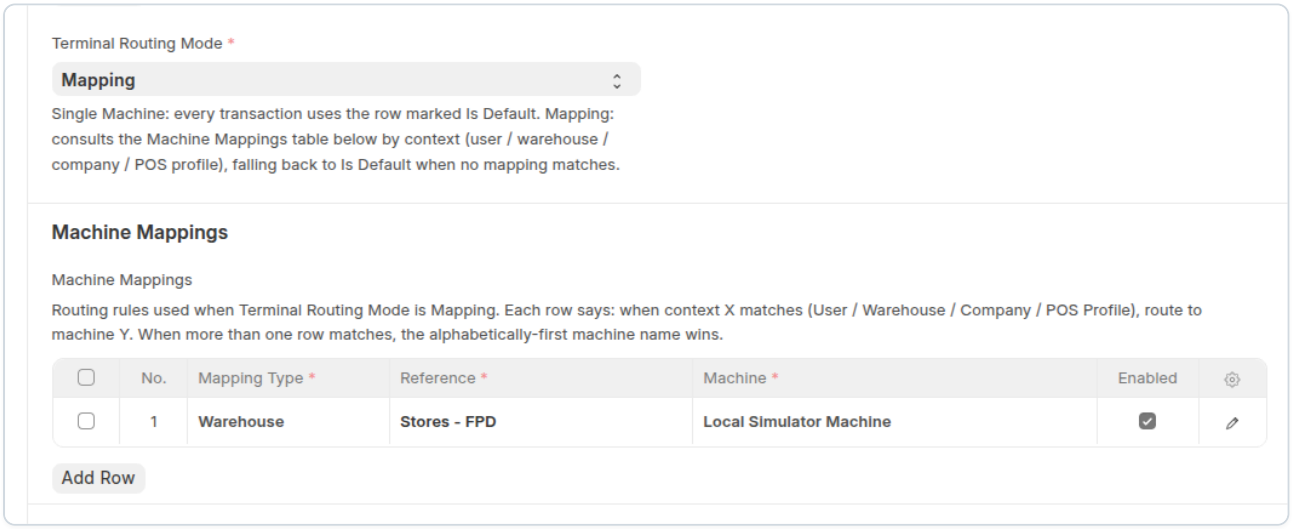
For the **Send Payment Link** flow you also need to tell the app where to find the customer's email and mobile:



Either point at a field directly on the source doc (**Direct Field**), or follow a link to another doctype (**Linked DocType**) — the most common setup is `customer_address → Address → phone` for mobile, `contact_email` direct for email.

Optional — multi-terminal routing

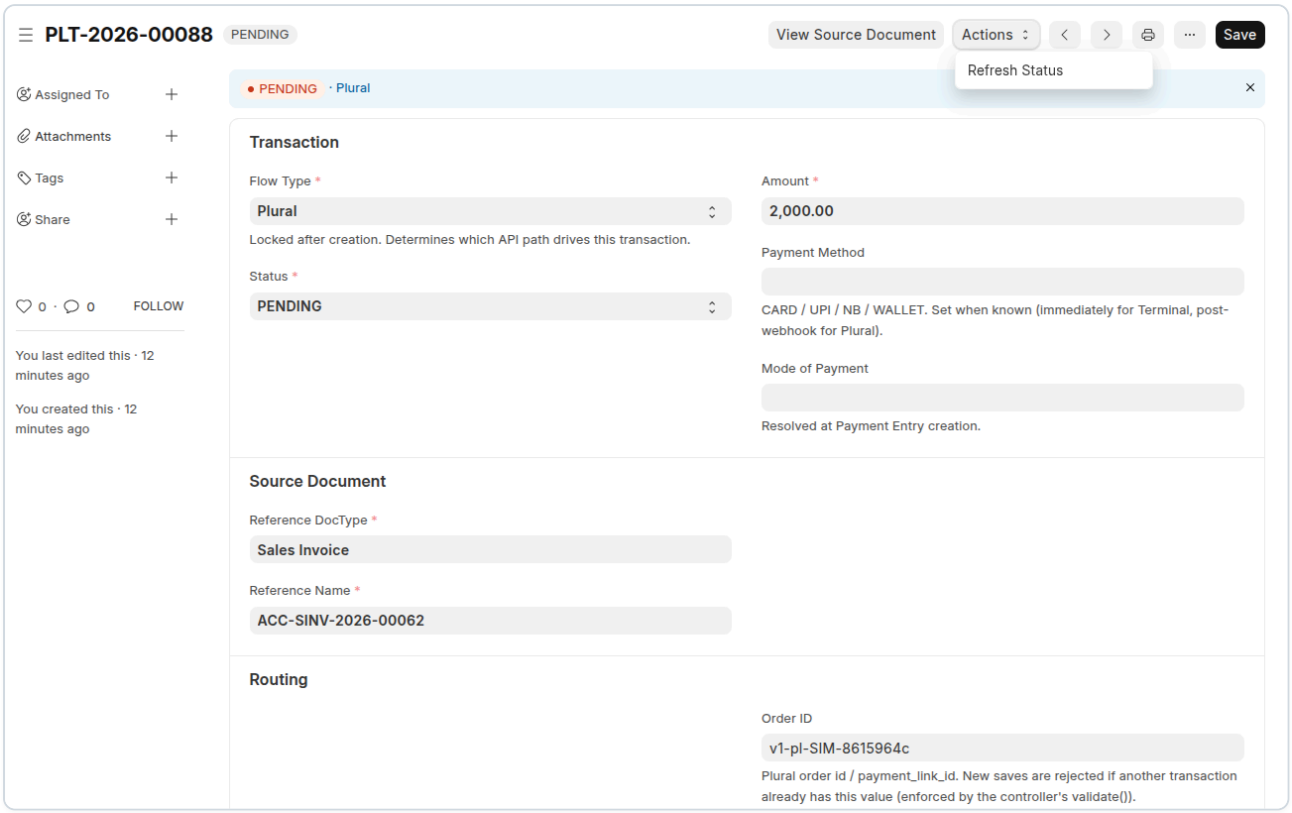
Have two or more terminals (one per till, one per warehouse)? Switch **Terminal Routing Mode** to `Mapping` and add rules:



The app picks the right terminal automatically based on who is at the till, what warehouse is on the invoice, or what POS profile is active. No rule matches? Falls back to the default machine.

Track every payment

Every payment attempt — terminal, link, retry — lands on one **Pinelabs Transaction** record. Status moves through `INITIATED → PENDING → SUCCESS / FAILED / CANCELLED`. The record carries the full request and response payloads for debugging.



Click **Actions** → **Refresh Status** to ask Pine Labs for the latest state on demand. Otherwise a 1-minute scheduler does it for you in the background.

For developers — call the same flows from your own code

Three Python entry points (whitelisted, callable from JS or REST):

```
pinelabs_fin.api.terminal.initiate_terminal_payment(reference_doctype, reference
pinelabs_fin.api.plural.initiate_payment_link(reference_doctype, reference_name,
pinelabs_fin.api.terminal.check_terminal_status(transaction_name)
```

Plus two `frappe.pinelabs.*` JS helpers that wrap the API and open the same UI the built-in buttons use:

```
frappe.pinelabs.start_terminal_payment(args, options);  // → "Waiting for termin
frappe.pinelabs.start_payment_link(args, options);      // → "Payment Link Gener
```

Full API contract + custom-button example: see guide.md section 5.

Documentation

What you want	Where to go
Install + configure step by step, with screenshots	**guide.md**
API contract for custom buttons	**guide.md §5**
Marketplace readiness, security notes, internal architecture	**PINELABS_FIN_GUIDE.txt**

License

MIT — see license.txt.

Support

Open a GitHub issue. Include the Pinelabs Transaction id of the affected payment and any `[PineLabs]` / `[PineLabs PayByLink]` lines from your `bench start` terminal.

Built with



· by


Finstein